ITSM/Kanban MVP — Descripción Técnica Completa

> **Documento de referencia para evaluación e incorporación del módulo a otros proyectos**

> Módulo de Incidencias con gestión ITSM estricta sobre visualización Kanban ágil

> Versión del módulo: 0.1.0 · Fecha: 2026-10-23

---

1. Resumen ejecutivo

Este módulo implementa un **Módulo de Incidencias** que reconcilia dos entidades que normalmente entran en conflicto:

- **La disciplina del ITSM**, que exige control formal: no cualquier cambio de estado es válido, cada movimiento tiene una justificación, un actor responsable y un rastro auditable inalterable.
- **La agilidad de Kanban**, que exige fluidez visual: el usuario arrastra una tarjeta y espera un resultado inmediato, sin recargas de página ni fricción.

La conciliación se logra con una idea central: **la interfaz es libre, el servidor es el autoridad**. El tablero permite mover la tarjeta visualmente en cualquier dirección, pero el servidor valida cada movimiento contra una matriz de reglas persistida en base de datos. Si la regla no se cumple, el cambio se rechaza y la tarjeta retorna a su posición original.

El resultado es un sistema donde **el compliance no se opone a la/usabilidad**, sino que la habilita: el agente mueve la tarjeta con naturalidad y, si el proceso exige justificación (por ejemplo, cerrar un ticket), el sistema la exige de forma automática y la registra.

1.1 Métricas verificadas del código

Capa	Archivos	Líneas
`app/models/`	8	476
`app/services/`	2	189
`app/api/`	7	440
`app/core/`	4	145
`app/db/`	3	254
`app/tasks/`	2	139
`app/templates/`	5	341
`tests/`	1	179
Total Python	—	**1.708**
Total HTML/Jinja2	—	**341**

Entidades de base de datos: 7 tablas · **Endpoints:** 15 · **Tareas asíncronas:** 4 · **Pruebas automatizadas:** 4 (todas en verde)

---

2. Problema que resuelve

En el soporte de incidencias se observa una fricción recurrente entre dos flujos de trabajo histórico:

Flujo ITSM tradicional (formal pero rígido). El técnico llena un formulario, espera aprobación, el ticket avanza por estados predefinidos. El Kanban se pierde porque el estado real del trabajo técnico no cabe en el formulario. La información del tablero es siempre obsoleta.

Flujo Kanban puro (fluido pero anárquico). Cualquiera mueve cualquier tarjeta a cualquier columna. Nadie sabe quién cambió el estado, cuándo ni por qué. No hay control de permisos, no hay SLA verificable, y auditar qué pasó con un ticket requiere reconstruirlo desde correos y chats. Cuando hay una auditoría de cumplimiento o post mortem, el equipo no puede responder.

Este módulo adopta la postura de que **la trazabilidad no puede ser un trámite posterior**, sino una consecuencia estructural del diseño. La tabla de auditoría no es una funcionalidad opcional que se activa cuando alguien la necesita: es una consecuencia inevitable de mover un ticket, porque el movimiento y su registro se escriben en la misma transacción de base de datos.

2.1 Ejemplo del flujo resuelto

```
Usuario arrastra «Login falla 500» de [Pendiente] a [En revisión]

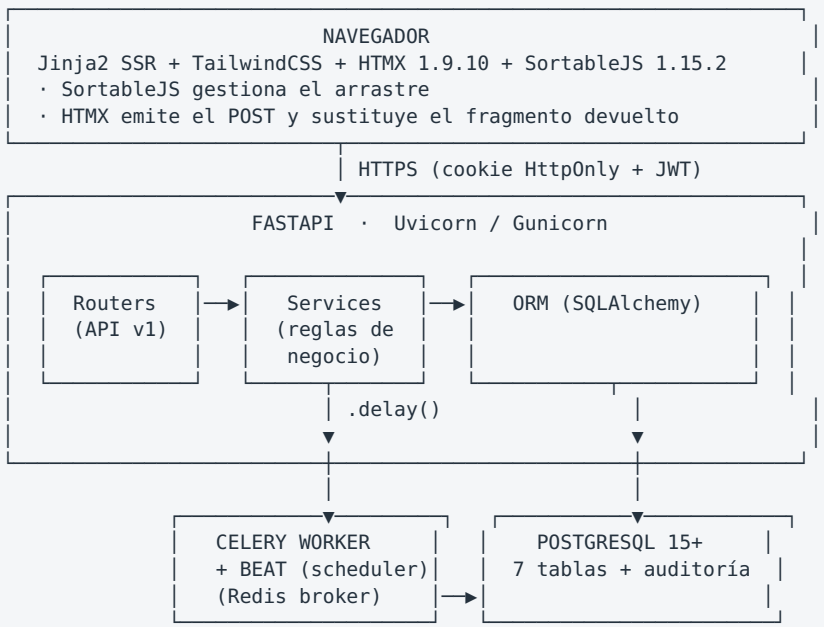
1. SortableJS detecta el soltar → lee ticket_id=42, estado_destino=4
2. HTMX envía POST /api/v1/kanban/tickets/42/mover {estado_id: 4, posicion: 0}
3. FastAPI consulta PostgreSQL por una regla Transicion(origen=1, destino=4)
4. Regla encontrada: «Enviar a revisión», requiere rol AGENTE
5. Usuario (AGENTE) cumple el rol → válido
6. UPDATE tickets SET estado_id=4 WHERE id=42
7. INSERT auditoria_eventos (tipo=ESTADO_CAMBIADO, actor='agente1',
       valor_anterior='{"estado":"Pendiente"}',
       valor_nuevo='{"estado":"En revisión"}')
8. Celery .delay() → recalcular_sla_ticket(42) + notificar_cambio_estado(...)
9. COMMIT (transacción única: paso 6 y 7 son atómicos)
10. Retorna solo el fragmento HTML de la tarjeta → HTMX reemplaza el nodo, sin recargar
```

Si en el paso 3 no existiera la regla `1→4`, el servidor responde `422` con el encabezado `HX-Reswap: none`. HTMX no ejecuta el intercambio, la tarjeta permanece visible en su columna original y una barra de error explica por qué.

---

3. Arquitectura

3.1 Vista de componentes



3.2 Patrón de flujo de datos

El sistema implementa un flujo de **validate-then-persist-then-notify** donde la validación precede siempre a la persistencia y ambas ocurren dentro de la misma transacción:

```
Validación (read-only)
    ├─ ¿Existe el ticket?                    → 404
    ├─ ¿Existe y está activo el destino?     → 422
    ├─ ¿Existe regla Transicion(origen,destino)? → 422
    ├─ ¿El rol del usuario satisface el mínimo?   → 403
    └─ ¿Si requiere_comentario, viene texto?      → 422
                  │  (si algo falla: NADA se escribe)
                  ▼
Persistencia (transacción atómica)
    ├─ UPDATE tickets (estado_id, posicion, updated_at)
    └─ INSERT auditoria_eventos              ← OBLIGATORIO
                  │
                  ▼
Notificación asíncrona (fuera de la transacción)
    └─ Celery .delay() → recálculo SLA + notificación
                  │
                  ▼
Respuesta (fragmento HTML, no página completa)
```

Esta separación garantiza que un rechazo nunca deja efectos secundarios parciales: si la validación falla, no hay escritura de ningún tipo.

---

4. Modelo de datos

El modelo se organiza alrededor de **siete tablas** que permanecen conformas a tres capas de responsabilidad.

4.1 Capa de identidad y permisos

Tabla `usuarios`

Columna	Tipo	Restricciones
`id`	INTEGER	PK, autoincrement
`username`	VARCHAR(64)	UNIQUE, INDEX, NOT NULL
`email`	VARCHAR(128)	UNIQUE, INDEX, NOT NULL
`nombre_completo`	VARCHAR(128)	NOT NULL, default `''`
`password_hash`	VARCHAR(255)	NOT NULL (hash bcrypt, nunca texto plano)
`rol`	ENUM(20)	NOT NULL, default `SOLICITANTE`
`activo`	BOOLEAN	NOT NULL, default `true`
`created_at` / `updated_at`	TIMESTAMPTZ	NOT NULL, server default / ON UPDATE

Enum `RolUsuario` — cinco roles con jerarquía implícita de privilegio:

Rol	Nivel	Descripción funcional
`OBSERVADOR`	0	Solo lectura. No puede transicionar nada.
`SOLICITANTE`	1	Crea tickets. No avanza el flujo de trabajo.
`AGENTE`	2	Ejecuta el trabajo: iniciar, bloquear, resolver.
`AGENTE_SENIOR`	3	Operaciones de cierre y cancelación, que requieren justificación.
`ADMINISTRADOR`	4	Acceso total. Única figura con autoridad para cerrar.

La jerarquía no está codificada en el esquema sino resuelta en tiempo de ejecución por el método `Transicion.usuario_tiene_permiso()`, que compara el nivel del rol del usuario contra el mínimo requerido por la transición. Esto permite **subir o bajar el privilegio de una transición editando una fila de base de datos**, sin tocar código.

Propiedades de conveniencia en el modelo: `Usuario.es_admin` y `Usuario.puede_cerrar_ticket` encapsulan las dos consultas de permiso más frecuentes.

4.2 Capa de ciclo de vida

Tabla `estados`

Representa simultáneamente el **estado ITSM** y la **columna del tablero Kanban**. No hay dos conceptos separados: cada estado es una columna con un orden, un color y una posición.

Columna	Tipo	Función
`nombre`	VARCHAR(64) UNIQUE	Identificador legible
`orden`	INTEGER	Posición izquierda→derecha en el tablero
`categoria`	ENUM(20)	Semántica funcional del estado
`color`	VARCHAR(7)	Color HEX de la cabecera de columna
`es_inicial` / `es_final`	BOOLEAN	Marcadores del ciclo de vida

Enum `CategoriaEstado` — agrupa estados que comparten comportamiento aunque su nombre varíe:

Categoría	Significado
`PENDIENTE`	Backlog, trabajo no iniciado
`EN_PROGRESO`	Trabajo activo en curso
`BLOQUEADO`	En espera por dependencia externa
`RESUELTO`	Trabajo terminado, pendiente de verificación
`CERRADO`	**Terminal.** Ciclo de vida concluido
`CANCELADO`	**Terminal.** Desestimado

El campo `es_final` tiene consecuencias funcionales reales: las tareas de Celery lo consultan para marcar el SLA como cumplido automáticamente (`recalcular_sla_ticket` línea 49) sin necesidad de esperar al vencimiento.

Tabla `transiciones` — el corazón del control ITSM

Esta tabla es la pieza que distingue este módulo de un Kanban convencional. Cada fila **declara qué movimiento es legal**.

Columna	Tipo	Función
`origen_id`	FK → `estados.id` ON DELETE CASCADE	Estado de partida
`destino_id`	FK → `estados.id` ON DELETE CASCADE	Estado de llegada
`nombre`	VARCHAR(64)	Etiqueta de la acción ("Iniciar", "Cerrar")
`rol_requerido`	ENUM(20)	Privilegio mínimo exigido
`requiere_comentario`	BOOLEAN	Si exige justificación escrita
`dispara_recalculo_sla`	BOOLEAN	Si debe encolar tarea asíncrona
`activo`	BOOLEAN	Interruptor para desactivar sin borrar

Restricción de unicidad: `UNIQUE (origen_id, destino_id)` garantiza que no existan reglas duplicadas ni ambiguas para el mismo par de estados. Esto es relevante porque `mover_ticket()` usa `.first()` para localizar la regla; sin la restricción, el comportamiento sería no determinista.

Matriz de transiciones sembrada por defecto (10 reglas):

Origen	→ Destino	Rol mínimo	Comentario	Nombre
Pendiente	En curso	AGENTE	No	Iniciar
Pendiente	Cancelado	AGENTE	**Sí**	Cancelar
En curso	Bloqueado	AGENTE	**Sí**	Bloquear
En curso	En revisión	AGENTE	No	Enviar a revisión
En curso	Pendiente	AGENTE	**Sí**	Devolver
Bloqueado	En curso	AGENTE	No	Desbloquear
Bloqueado	Cancelado	AGENTE_SENIOR	**Sí**	Cancelar (bloq.)
En revisión	En curso	AGENTE	**Sí**	Devolver a curso
En revisión	Cerrado	AGENTE_SENIOR	**Sí**	Cerrar
En revisión	Cancelado	AGENTE_SENIOR	**Sí**	Cancelar (rev.)

Lectura de negocio de la matriz: el flujo happy-path es `Pendiente → En curso → En revisión → Cerrado`. Cerrar y cancelar exigen siempre justificación escrita y privilegio AGENTE_SENIOR. Las flechas de retroceso (Devolver, Desbloquear) existen deliberadamente: un ITSM rígido que solo permite avanzar obliga a los técnicos a "hackear" el sistema. Aquí el retroceso es una transición de primera clase, auditada como cualquier otra.

Tabla `tickets`

Columna	Tipo	Notas
`codigo`	VARCHAR(32) UNIQUE INDEX	Identificador de negocio legible (`TKT-00001`)
`titulo`	VARCHAR(255) NOT NULL	Resumen en una línea
`descripcion`	TEXT	Detalle libre
`estado_id`	FK → `estados.id` **RESTRICT**	La columna actual del Kanban
`prioridad`	ENUM(10)	`BAJA` · `MEDIA` · `ALTA` · `CRITICA`
`creador_id`	FK → `usuarios.id` RESTRICT	Quién reporta
`asignado_id`	FK → `usuarios.id` SET NULL	Quién atiende. NULL si desasignado
`posicion`	INTEGER	Orden manual dentro de la columna
`fecha_vencimiento_sla`	TIMESTAMPTZ NULL	Calculado por Celery
`sla_cumplido`	BOOLEAN NULL	Ternario: `NULL`=sin calcular, `TRUE`=en plazo, `FALSE`=vencido
`metadata_json`	TEXT default `'{}'`	Extensibilidad sin migración

Decisiones de integridad relevantes:

- `estado_id` usa `ON DELETE RESTRICT`: es imposible borrar un estado que tenga tickets asociados. Esto protege la integridad del historial.
- `asignado_id` usa `ON DELETE SET NULL`: desasignar un usuario no destruye tickets. Se prefiere perder la asignación antes que el registro.

- Índice compuesto `ix_tickets_estado_orden (estado_id, posicion)`: está calibrado exactamente para la consulta del tablero, que siempre pide "los tickets de esta columna ordenados por posición". Sin este índice, cada renderizado del Kanban degrada a un escaneo secuencial conforme crece el histórico.

El ternario de `sla_cumplido` merece atención: `NULL` no significa "cumplió", significa "aún no se ha calculado". Esta distinción entre "no sabemos" y "sabemos que no cumplió" es fundamental en la gestión de SLA y se refleja directamente en la interfaz, donde la tarjeta muestra tres estados visuales distintos: *Sin calcular* (gris), *En plazo* (verde), *Vencido* (rojo).

4.3 Capa de trazabilidad y soporte

Tabla `auditoria_eventos` — registro append-only

Columna	Tipo	Función
`ticket_id`	FK → `tickets.id` ON DELETE CASCADE	Ticket afectado
`tipo`	ENUM(30) INDEX	Naturaleza del evento
`actor_id`	FK → `usuarios.id` SET NULL	Autor (NULL si fue un proceso automático)
`actor_username`	VARCHAR(64)	**Desnormalizado deliberadamente**
`valor_anterior`	TEXT	Estado previo, JSON serializado
`valor_nuevo`	TEXT	Estado nuevo, JSON serializado
`comentario`	TEXT	Justificación escrita
`created_at`	TIMESTAMPTZ INDEX	Marca temporal

Enum `TipoEvento` — taxonomía de 8 eventos auditables:

Evento	Cuándo se emite
`CREADO`	Alta del ticket
`ESTADO_CAMBIADO`	Movimiento validado en el Kanban
`REASIGNADO`	Cambio de responsable
`PRIORIDAD_CAMBIADA`	Reclasificación por severidad
`COMENTARIO_AGREGADO`	Interacción del usuario
`ADJUNTO_AGREGADO`	Carga de evidencia
`SLA_CALCULADO`	Tarea Celery recalcula el plazo
`NOTIFICACION_ENVIADA`	Proceso automático notifica

Por qué `actor_username` está desnormalizado. Es una decisión deliberada que merece justificación explícita. Cuando un usuario se elimina de la tabla `usuarios` (`actor_id` pasa a NULL por `ON DELETE SET NULL`), el registro de auditoría **conserva el nombre de quien actuó**. En una auditoría de cumplimiento, «el usuario #47 movió este ticket» es inservible dos años después si #47 ya no existe. La desnormalización compra integridad del registro a costa de una anomalía de actualización controlada, y es el compromiso correcto para tablas append-only.

Modelo strictly append-only. El modelo declara sus campos sin `updated_at` y su relación con cascada `all, delete-orphan`, lo que impide actualizaciones accidentales desde la capa de aplicación. No existe endpoint que modifique o elimine un evento.

Tabla `sla_politicas`

Columna	Tipo	Función
`nombre`	VARCHAR(64) UNIQUE	Identificador de la política
`prioridad`	ENUM(10) INDEX	Prioridad a la que aplica
`minutos_resolucion`	INTEGER	Plazo máximo en minutos
`es_default`	BOOLEAN	Política activa para esa prioridad
`activo`	BOOLEAN	Interruptor

Políticas sembradas:

Política	Prioridad	Plazo
SLA Crítica	`CRITICA`	4 horas
SLA Alta	`ALTA`	24 horas
SLA Media	`MEDIA`	72 horas
SLA Baja	`BAJA`	168 horas (7 días)

El flag `es_default` permite mantener **múltiples políticas por prioridad** (por ejemplo, por cliente o por contrato) y seleccionar cuál aplica sin duplicar código.

Tabla `catalogo_items` — base para mantenedores futuros

Tabla única que hospeda todos los catálogos del sistema, discriminados por la columna `tipo`. Este diseño evita crear una tabla nueva por cada concepto configurable.

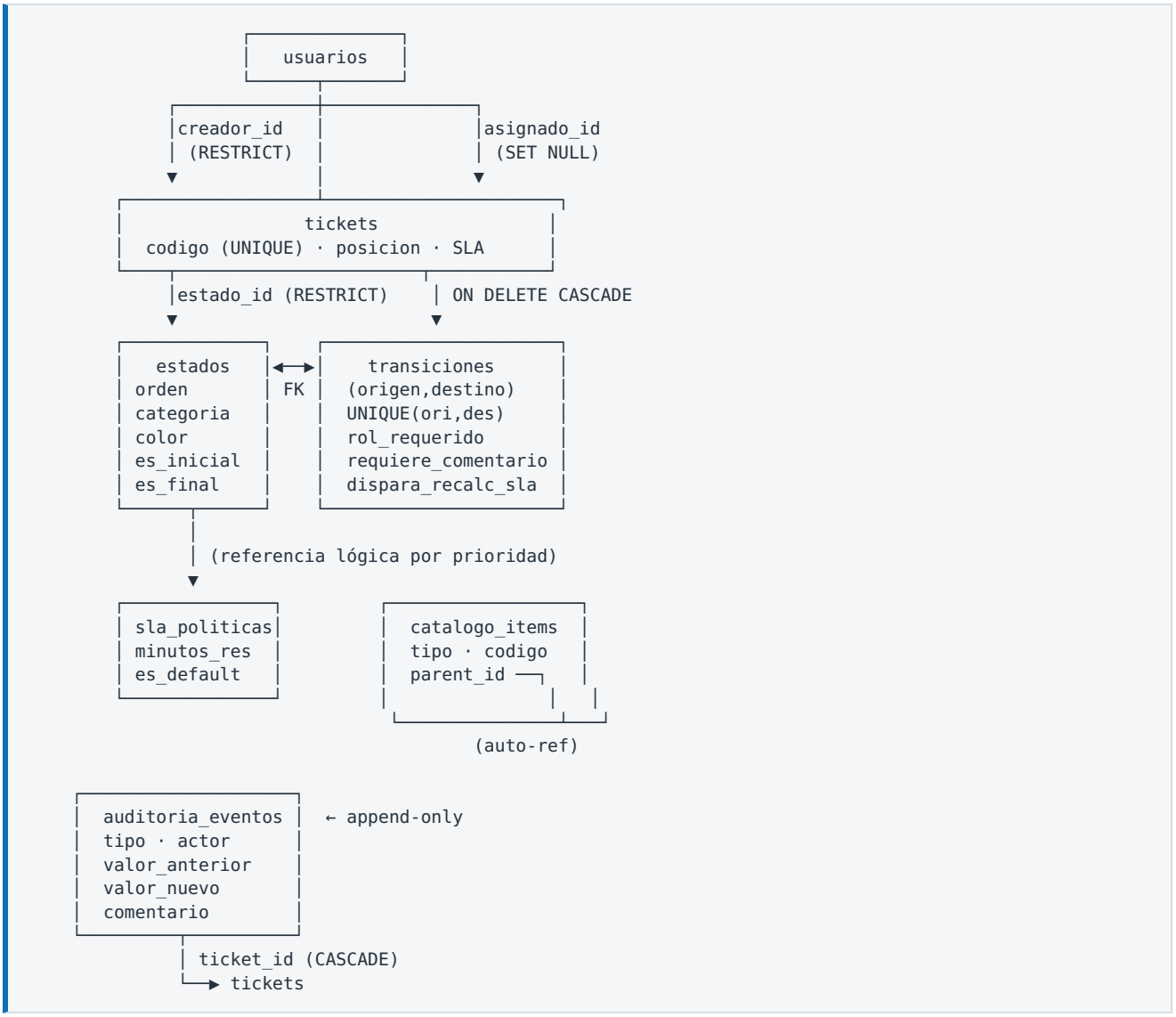
Campo	Función
`tipo`	`TIPO_TICKET` · `CATEGORIA` · `ETIQUETA` · `MODULO`
`codigo`	Código corto de negocio (`INC`, `HW`, `AUTH`)
`nombre` / `descripcion`	Etiqueta visible y glosa
`color`	Color HEX para representación visual
`parent_id`	FK autorreferenciada para **jerarquías**
`orden`	Posición en el mantenedor

Restricción: `UNIQUE (tipo, codigo)` — el mismo código puede repetirse en catálogos distintos (un `HW` de categoría y un `HW` de módulo coexisten sin conflicto).

Auto-referencia. `parent_id` con `ON DELETE SET NULL` habilita árboles de jerarquía arbitrariamente profundos (Categoría → Subcategoría → Sub-subcategoría). El campo `activo` en toda la tabla —presente en esta misma tabla — permite desactivar ítems sin borrarlos, preservando la integridad referencial de tickets históricos que los|Marcaban.

Catálogos sembrados (11 ítems): 4 tipos de ticket (Incidencia, Requerimiento, Bug, Consulta), 4 categorías (Hardware, Software, Redes, Seguridad), 3 módulos (Autenticación, API, Interfaz).

4.4 Diagrama entidad-relación



5. Reglas de negocio ITSM

Las reglas viven en la capa de servicios (`app/services/kanban_service.py`), no en los endpoints ni en la interfaz. Esta separación permite invocar la misma lógica desde un futuro API REST, desde una tarea programada o desde un proceso de importación masiva.

5.1 Función `mover_ticket()` —núcleo del sistema

Es la única vía por la que un ticket puede cambiar de estado. Firma:

```
mover_ticket(db, ticket_id, estado_destino_id, usuario,
             nueva_posicion=0, comentario="") -> dict
```

Secuencia de validación (ejecutada en orden, cortando en el primer fallo):

#	Validación	Código de error	Razón
1	El ticket existe	404	Recurso inexistente
2	El ticket tiene estado asociado	500	Integridad referencial rota
3	El destino existe y está activo	422	Columna inválida o desactivada
4	Existe regla `Transicion(origen, destino)`	422	Movimiento no contemplado en el flujo
5	El rol satisface `rol_requerido`	403	Privilegio insuficiente
6	Si `requiere_comentario`, viene texto	422	Falta justificación

Códigos HTTP diferenciados por causa: la distinción entre 403 (*no autorizado*) y 422 (*transición mal formada*) es deliberada. Un 403 indica un problema de permisos del usuario —debe mostrársele directamente. Un 422 indica que la operación no tiene sentido en el flujo de trabajo, independientemente de quién la intente.

Efectos laterales tras validación exitosa:

```
# 1. Mutación del ticket
ticket.estado_id = estado_destino.id
ticket.posicion  = nueva_posicion
ticket.updated_at = now(UTC)

# 2. Registro de auditoría — OBLIGATORIO, misma transacción
db.add(AuditoriaEvento(
    tipo=TipoEvento.ESTADO_CAMBIADO,
    actor_username=usuario.username,      # desnormalizado
    valor_anterior=json.dumps({...}),
    valor_nuevo=json.dumps({...}),
    comentario=comentario,
))

# 3. Notificación asíncrona — tolerante a fallos
if transicion.dispara_recalculo_sla:
    try:
        recalcular_sla_ticket.delay(ticket.id)
        notificar_cambio_estado.delay(ticket.id, nombre_origen, estado_destino.nombre)
    except Exception:
        pass   # broker caído no impide el movimiento

db.commit()   # ← pasos 1 y 2 son atómicos
```

Decisión de diseño sobre Celery tolerante a fallos. Las llamadas `.delay()` están envueltas en `try/except` que absorbe la excepción. Un broker Redis caído no debe impedir que un agente mueva un ticket. La degradación

preferida es **perder el recálculo de SLA** antes que **bloquear el trabajo del equipo**. La consecuencia —un SLA sin recalcular— es reversible: el job de Celery Beat reintenta cada 5 minutos.

5.2 Matriz de privilegios

`Transicion.usuario_tiene_permiso()` implementa la resolución de jerarquía:

```
nivel_usuario  >=  nivel_requerido  →  permitido
```

Usuario \ Requerido	CUALQUIERA (0)	SOLICITANTE (1)	AGENTE (2)	AGENTE_SENIOR (3)	ADMINISTRADOR (4)
OBSERVADOR (0)	Si	No	No	No	No
SOLICITANTE (1)	Si	Si	No	No	No
AGENTE (2)	Si	Si	Si	No	No
AGENTE_SENIOR (3)	Si	Si	Si	Si	No
ADMINISTRADOR (4)	Si	Si	Si	Si	Si

La matriz es un producto cartesiano de niveles, no una colección de reglas exóticas: cualquier rol con nivel suficiente satisface cualquier requisito con nivel suficiente. Esto elimina la necesidad de enumerar combinaciones permitidas.

---

6. Procesamiento asíncrono

6.1 Configuración Celery

```
celery_app = Celery(
    "itsm_kanban",
    broker=settings.celery_broker_url,      # Redis DB 1
    backend=settings.celery_result_backend, # Redis DB 2
    include=["app.tasks.kanban_tasks"],
)
```

Redis usa **tres bases de datos lógicas separadas** (0 para aplicación, 1 para broker, 2 para resultados). Esto evita colisiones de claves entre el estado de la aplicación y los mensajes de la cola, un problema habitual cuando todo comparte el mismo namespace.

Parámetros de robustez aplicados:

Parámetro	Valor	Justificación
`task_acks_late`	`true`	El broker reintent si el worker muere a mitad de tarea
`task_reject_on_worker_lost`	`true`	Devuelve el mensaje a la cola en pérdida de worker
`worker_prefetch_multiplier`	`1`	Distribución justa: una tarea por worker a la vez
`task_soft_time_limit`	60 s	Interrupción limpia con excepción
`task_time_limit`	120 s	Terminación forzada del proceso colgado
`broker_connection_retry_on_startup`	`true`	El worker espera a Redis en despliegues en frío
`task_track_started`	`true`	Permite distinguir *iniciada* de *en cola*
`timezone` / `enable_utc`	`UTC`	Normalización horaria entre procesos

`worker_prefetch_multiplier = 1` merece comentario: con el valor por defecto (4), un worker reserva cuatro tareas de una vez, lo que en un despliegue con varios workers produce asimetría en la distribución — un worker puede monopolizar un lote mientras otro espera vacío. Fijarlo en 1 reparte el trabajo uniformemente.

6.2 Tareas implementadas (4)

`recalcular_sla_ticket(ticket_id)` —cálculo de plazo individual

Disparada por cada movimiento de ticket. Lógica:

1. Busca la política SLA default para la prioridad actual del ticket.
2. Si el estado es **terminal** (`es_final`), marca `sla_cumplido = True` sin más cálculo.
3. Si no tiene vencimiento asignado, lo calcula como `now() + minutos_resolucion`.
4. Compara con la hora actual para determinar cumplimiento.
5. **Registra un evento `SLA_CALCULADO` en auditoría**, firmado por `actor_username="system"`.

El paso 5 cierra el ciclo de trazabilidad: los cambios de SLA hechos por procesos automáticos también quedan auditados, con el sistema identificado como actor.

`recalcular_slas_vencidos()` — barrido programado

Ejecutada cada 5 minutos por Celery Beat. Localiza tickets no terminales y **encola un recálculo individual por cada uno**, en lugar de recalcularlos todos en un único proceso. Esta decisión mantiene el trabajo paralelo y evita que un lote grande bloquee el worker durante minutos.

`notificar_cambio_estado(ticket_id, anterior, nuevo)`

Registra un evento `NOTIFICACION_ENVIADA`. En el MVP el canal de entrega es la propia auditoría; el punto de extensión está marcado explícitamente en el docstring para conectar SMTP, Slack o webhook.

`notificar_cambios_pendientes()` — marcador de posición

Ejecutada cada hora. Actualmente es un *stub* documentado como tal.

6.3 Planificador Celery Beat

```
celery_app.conf.beat_schedule = {
    "recalcular-slas-vencidos":     {"task": "...recalcular_slas_vencidos",     "schedule":
crontab(minute="*/5")},
    "notificar-cambios-pendientes": {"task": "...notificar_cambios_pendientes", "schedule":
crontab(minute=0)},
}
```

Reconciliación SLA cada 5 minutos · Notificaciones cada hora en punto.

---

7. Capa de API

7.1 Superficie de endpoints (15 rutas)

Método	Ruta	Descripción	Render	Auth
`GET`	`/`	Redirige a `/kanban` (303)	—	No
`GET`	`/health`	Health check textual (`OK`)	HTML	No
`GET`	`/salud`	Health check en JSON	JSON	No
`GET`	`/auth/login`	Formulario de acceso	HTML	No
`POST`	`/auth/login`	Autenticación → cookie JWT + redirect (303)	—	No
`POST`	`/auth/logout`	Elimina cookie → redirect	—	No
`GET`	`/auth/me`	Perfil del usuario autenticado	JSON	Sí
`GET`	`/kanban`	Tablero completo	HTML	Sí
`POST`	`/api/v1/kanban/tickets/{id}/mover`	**Movimiento de tarjeta (endpoint HTMX)**	HTML	Sí
`GET`	`/api/v1/kanban/columnas`	Columnas en JSON para refresco	JSON	Sí
`POST`	`/tickets/nuevo`	Alta de ticket + auditoría `CREADO`	—	Sí
`GET`	`/tickets/{id}/auditoria`	Historial del ticket	HTML	Sí
`GET`	`/api/v1/catalogos/estados`	Estados + transiciones permitidas	JSON	Sí
`GET`	`/api/v1/catalogos/sla`	Políticas SLA activas	JSON	Sí
`GET`	`/api/v1/catalogos/items?tipo=`	Ítems de catálogo filtrables	JSON	Sí

Todos los endpoints salvo los tres de salud y los dos de acceso están protegidos por `Depends(get_current_user)`.

7.2 El endpoint HTMX de movimiento

```
@router.post("/api/v1/kanban/tickets/{ticket_id}/mover", response_class=HTMLResponse)
```

Protocolo de éxito (2xx). Retorna exclusivamente el fragmento Jinja2 `kanban/_tarjeta.html` renderizado con el ticket actualizado. HTMX lo inyecta con `swap: outerHTML` sobre el nodo original. La interfaz se actualiza; el resto del tablero permanece intacto; no hay recarga de página.

Protocolo de fallo (403 / 422). Retorna el mensaje de error como fragmento junto con dos encabezados de control:

```
headers={
    "HX-Reswap":   "none",              # instruye a HTMX: NO intercambies el DOM
    "HX-Trigger":  "kanban-error",      # dispara un evento para la interfaz
}
```

`HX-Reswap: none` es el mecanismo que cumple el requisito funcional de «la tarjeta vuelve a su origen»: sin intercambio, el nodo que HTMX iba a reemplazar permanece donde el navegador lo dejó. Combinado con el evento `kanban-error`, que muestra el mensaje en una barra temporal, el usuario recibe el motivo del rechazo sin ruido visual ni recargas de página.

Códigos de estado propagados: el endpoint preserva el código del error (403 o 422) en lugar de normalizarlo a 200, de modo que cualquier monitoreo externo puede distinguir causas.

7.3 Autenticación y sesión

Mecanismo. Login por formulario tradicional. El servidor verifica bcrypt, genera un JWT firmado con HS256 e incluye tres claims: `sub` (id del usuario), `iat`, `exp`, más claims extra (`rol`, `username`). El token se almacena en cookie `access_token` con atributos:

Atributo	Valor	Justificación
`httponly`	`true`	Inaccesible desde JavaScript → mitiga XSS
`samesite`	`lax`	(csrf básico) previene envío en navegación cruzada
`secure`	`false` *(hardcoded)*	**Debe ser `true` bajo HTTPS en producción**
`max_age`	`jwt_expiration_minutes × 60`	480 min por defecto (8 horas)

Resolución del usuario (`get_current_user`): acepta indistintamente la cookie o el encabezado `Authorization: Bearer`, lo que habilita el mismo backend tanto para sesión de navegador como para clientes API. Tras decodificar el token, **revalida contra la base de datos** que el usuario exista y esté `activo`. Esto permite revocar el acceso de una cuenta desactivada sin esperar a que expire el token.

7.4 Hash de contraseñas

```
def _to_bcrypt_bytes(password: str) -> bytes:
    return password.encode("utf-8")[:72]
```

bcrypt trunca silenciosamente su entrada a 72 bytes; en versiones recientes esto **lanza excepción** en lugar de truncar. El módulo trunca explícitamente para conservar compatibilidad y evitar un fallo en runtime al registrar contraseñas largas o con caracteres multibyte. Coste: 12 rondas (`bcrypt.gensalt(rounds=12)`).

---

8. Capa de presentación

8.1 Filosofía de renderizado

Arquitectura **Renderizado en el servidor con respuesta parcial**. Jinja2 produce HTML completo en la navegación y fragmentos en las interacciones. No hay cliente SPA, ni bundler, ni `package.json`, ni paso de compilación.

Consecuencia operativa relevante: no existe artefacto de build. Un despliegue es un `git pull` y un reinicio de proceso. Esto elimina una categoría completa de fallos de despliegue y reduce el tiempo de reconstrucción a segundos.

Dependencias de frontend (todas por CDN):

Librería	Versión	Origen
TailwindCSS	CDN (JIT en navegador)	`cdn.tailwindcss.com`
HTMX	1.9.10	`unpkg.com`
SortableJS	1.15.2	`cdn.jsdelivr.net`

8.2 Integración SortableJS + HTMX

La integración de las dos librerías es donde reside la pieza más delicada del diseño, porque cada una cubre un aspecto distinto del problema:

- **SortableJS** resuelve el **mecanismo físico** del arrastre: hover, reordenamiento, animaciones, clases visuales durante el drag.
- **HTMX** resuelve el **ciclo de red**: emitir la petición y reemplazar el DOM con la respuesta del servidor.

El puente es el evento `onEnd` de SortableJS, que entrega `evt.item` (la tarjeta) y `evt.to` (la columna destino):

```
new Sortable(col, {
    group: 'kanban-tickets',
    animation: 150,
    ghostClass: 'dragging',
    onEnd: function (evt) {
        var ticketId = evt.item.dataset.ticketId;
        var estadoId = evt.to.dataset.estadoId;

        // Reordenamiento dentro de la misma columna: no requiere ida al servidor
        if (evt.from === evt.to && evt.oldIndex === evt.newIndex) return;

        htmx.ajax('POST', '/api/v1/kanban/tickets/' + ticketId + '/mover', {
            target: evt.item,     // nodo a reemplazar
            swap:   'outerHTML',   // ← el backend devuelve la tarjeta nueva
            values: { estado_id: estadoId, posicion: evt.newIndex, comentario: '' },
            headers: { 'HX-Request': 'true' }
        });
    }
});
```

Los datos viajan por atributos `data-*`, no por inspeccionar el DOM: `data-ticket-id` en cada tarjeta y `data-estado-id` en cada columna. Esto mantiene la lógica JavaScript desacoplada de la estructura visual y permite reordenar columnas o cambiar el diseño sin tocar el código de arrastre.

Optimización de tráfico: el reordenamiento dentro de la misma columna se resuelve con una guarda `if (evt.from === evt.to && evt.oldIndex === evt.newIndex) return;`. Priorizar el orden manual de tarjetas no es un cambio de estado, no requiere validación ITSM, y se descarta en el cliente sin generar tráfico de red.

8.3 Manejo de errores en cliente

```
document.body.addEventListener('htmx:beforeSwap', function (evt) {
    if (evt.detail.xhr.status >= 400) {
        evt.detail.shouldSwap = false;          // ≡ HX-Reswap: none
        var bar = document.getElementById('kanban-error-bar');
        bar.textContent = 'Error ' + evt.detail.xhr.status + ': ' + evt.detail.xhr.responseText;
        bar.classList.remove('hidden');
        setTimeout(function () { bar.classList.add('hidden'); }, 5000);
        setTimeout(function () { location.reload(); }, 1500);   // resincroniza estado
    }
});
```

La recarga diferida 1,5 segundos no es un accidente: la tarjeta *visualmente* permanece en su columna de destino durante esa ventana, dando al usuario la percepción de que el movimiento funcionó, antes de que el servidor restaure la verdad. Es una compensación deliberada de la latencia percibida, aunque implica que el requisito «sin recargar» se cumple parcialmente en el camino de error.

8.4 Fragmentos reutilizables

Plantilla	Propósito
`base.html`	Layout, navegación, CDN, sistema de diseño
`auth/login.html`	Pantalla de acceso (centrada, sin navbar)
`kanban/tablero.html`	Tablero: columnas, modal de alta, inicialización Sortable
`kanban/_tarjeta.html`	**Fragmento de tarjeta.** Usado en el render inicial *y* devuelto por el endpoint HTMX
`kanban/_auditoria.html`	Historial del ticket, cargado bajo demanda

`kanban/_tarjeta.html` cumple doble función: se incluye con `{% include %}` durante el render del tablero y se devuelve como respuesta del endpoint de movimiento. **Una sola definición de la tarjeta** garantiza que lo que el usuario ve tras arrastrar sea idéntico a lo que habría visto en una carga completa de página. Cualquier cambio futuro en el diseño de la tarjeta se propaga automáticamente a ambos contextos.

8.5 Componentes de la tarjeta

Elemento	Comportamiento
Código (`TKT-00001`)	Monoespaciado, gris, referencia de negocio
Badge de prioridad	Color por nivel: Crítica (rojo) · Alta (naranja) · Media (azul) · Baja (gris)
Título	Dos líneas como máximo (`line-clamp-2`)
Indicador SLA	Punto de color con tres estados: *Sin calcular* (gris) · *En plazo* (verde) · *Vencido* (rojo)
Botón de auditoría	Carga el historial bajo demanda vía `hx-get` sin salir de la tarjeta

El color de cada columna proviene del campo `color` de la tabla `estados` y se aplica con `style="background-color: {{ col.color }}"`. Reordenar columnas, renombrarlas o cambiar su color es una operación de base de datos, no una modificación de plantilla.

---

9. Seguridad

9.1 Controles implementados

Capa	Control	Implementación
Contraseñas	Hash bcrypt con sal, 12 rondas	`app/core/security.py`
Contraseñas	Truncado defensivo a 72 bytes	Evita excepción en bcrypt ≥ 4
Sesión	JWT HS256 con expiración de 8 h	Claims `sub`, `iat`, `exp`
Sesión	Cookie `HttpOnly` + `SameSite=Lax`	Inaccesible a JS, resistente a CSRF básico
Autorización	Matriz rol → privilegio por transición	`usuario_tiene_permiso()`
Autorización	Verificación de `activo` en cada request	Revocación inmediata
Entrada	Validación de enums con `HTTPException` 422	Prioridad, estado destino
Entrada	Recorte de cadenas a longitudes de columna	`titulo.strip()[:255]`
SQL	**Consultas parametrizadas por el ORM**	Sin concatenación de SQL en todo el código
Auditoría	Registro inmutable de toda mutación	Tabla append-only
Fuga de información	Mensajes de error sin detalle interno	Stack traces no expuestos

Ausencia total de concatenación SQL. Una revisión línea por línea del servicio y los endpoints confirma que no existe construcción dinámica de sentencias: todo acceso a datos pasa por el query builder de SQLAlchemy, que parametriza valores automáticamente.

9.2 Superficie de ataque conocida

Vector	Estado	Mitigación requerida en producción
CSRF	Parcial	`SameSite=Lax` mitiga el caso común. Requiere token CSRF para operaciones cross-origin
Cookie sobre HTTP	**Abierto**	`secure=False` hardcodeado. Debe parametrizarse
Fuerza bruta	**No mitigada**	Sin rate limiting ni bloqueo por intentos
SQL Injection	Cerrado	Parametrización total del ORM
XSS	Mitigado	Jinja2 autoescapa + `HttpOnly`
Escalada de privilegios	Cerrado	Validación por nivel de rol
Dependencias	Sin verificar	Ausencia de auditoría de vulnerabilidades (Dependabot/Snyk)

10. Configuración y despliegue

10.1 Variables de entorno

Gestionadas por Pydantic Settings v2 con validación en carga. El proceso **falla al arrancar** si falta una obligatoria, lo que impide desplegar con configuración incompleta.

Variable	Obligatoria	Default	Descripción
`SECRET_KEY`	**Sí**	—	Clave de firma JWT. Generar con `openssl rand -hex 32`
`DATABASE_URL`	**Sí**	—	Cadena de conexión PostgreSQL
`REDIS_URL`	No	`redis://localhost:6379/0`	Redis de aplicación
`CELERY_BROKER_URL`	No	`redis://localhost:6379/1`	Broker de Celery
`CELERY_RESULT_BACKEND`	No	`redis://localhost:6379/2`	Resultados de Celery
`JWT_ALGORITHM`	No	`HS256`	Algoritmo de firma
`JWT_EXPIRATION_MINUTES`	No	`480`	Duración de sesión (8 h)
`APP_NAME`	No	`ITSM Kanban MVP`	Nombre visible en la interfaz
`DEBUG`	No	`false`	Modo depuración
`PUBLIC_BASE_URL`	No	`http://localhost:8000`	URL pública
`BOOTSTRAP_ADMIN_USERNAME`	No	`admin`	Usuario inicial
`BOOTSTRAP_ADMIN_PASSWORD`	No	`admin123`	**Cambiar en producción**
`BOOTSTRAP_ADMIN_EMAIL`	No	`admin@example.com`	Correo inicial

Riesgo de configuración: `BOOTSTRAP_ADMIN_PASSWORD` tiene default `admin123` y se aplica automáticamente en el primer arranque. Si se despliega sin sobrescribirla, existe una cuenta administrativa con credencial publicada en el repositorio.

10.2 Inicialización de base de datos

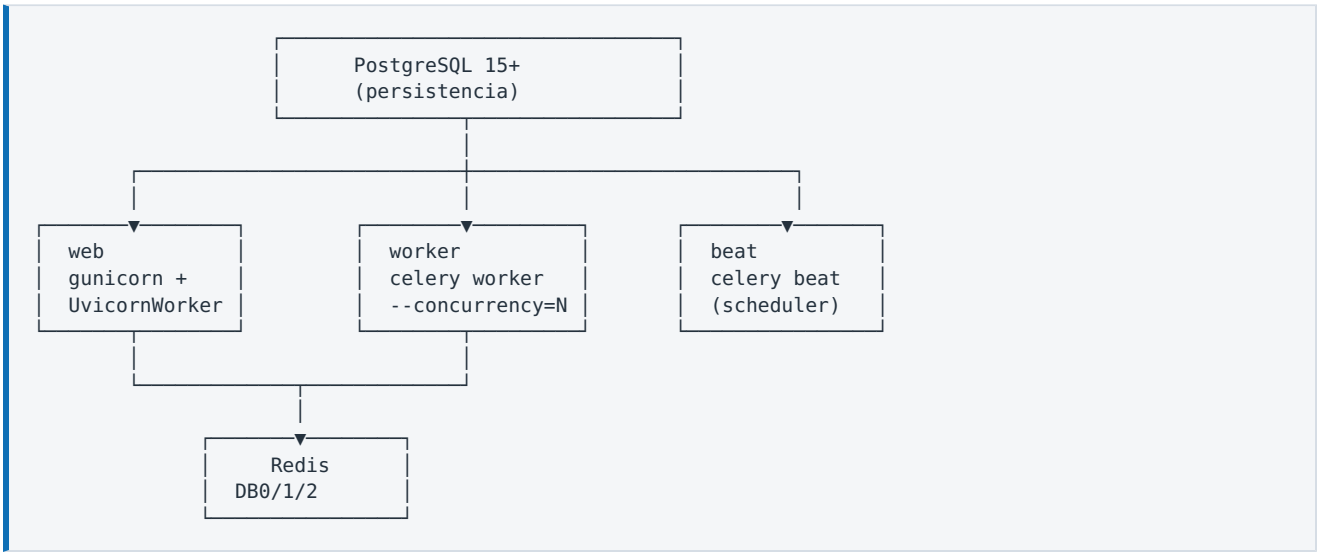
`python -m app.db.init_db` ejecuta una secuencia de pasos **idempotentes** (cada uno verifica si el recurso ya existe antes de crearlo):

Paso	Función	Resultado
1	`_crear_tablas()`	7 tablas vía `create_all()`
2	`_seed_admin()`	1 usuario administrador
3	`_seed_estados()`	6 estados del ciclo de vida
4	`_seed_transiciones()`	10 reglas de transición
5	`_seed_sla()`	4 políticas SLA
6	`_seed_catalogos()`	11 ítems de catálogo
7	`_seed_tickets_ejemplo()`	6 tickets, uno por estado

La idempotencia permite ejecutar el sembrado en cada arranque sin riesgo de duplicación —comportamiento construido en el hook `lifespan` de FastAPI, dentro de un `try/except` que registra pero no detiene el arranque.

10.3 Topología de despliegue

El módulo requiere **cuatro procesos** sobre infraestructura compartida:



Los tres procesos se lanzan con el mismo punto de entrada, que detecta su rol:

```
celery -A app.core.celery_app:celery_app worker --loglevel=info --concurrency=2
celery -A app.core.celery_app:celery_app beat   --loglevel=info
uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

Verificación de salud disponible en `/health` (texto) y `/salud` (JSON), útil como `healthcheckPath` en Railway, Fly.io o Kubernetes.

---

11. Pruebas automatizadas

4 pruebas unitarias sobre el servicio de negocio, todas en verde, ejecutadas sobre SQLite en memoria (`StaticPool`) para aislamiento completo y sin dependencias externas.

Prueba	Verifica
`test_transicion_legal_admin`	Un movimiento con regla existente y rol suficiente se ejecuta **y genera exactamente un evento de auditoría** con el `actor_username` correcto
`test_transicion_ilegal`	Saltarse una etapa del flujo (`Pendiente → Cerrado` sin regla) lanza `HTTPException` con código 422
`test_comentario_requerido`	Una transición con `requiere_comentario=True` falla sin comentario (422) y **tiene éxito** al proporcionarlo
`test_auditoria_obligatoria`	El recuento de eventos de auditoría se incrementa exactamente en uno por movimiento exitoso

Características de la suite:

- **Aislamiento:** base SQLite en memoria creada y destruida por prueba (`create_all` / `drop_all` en fixture).
- **Determinismo:** el fixture siembra un estado mínimo conocido (3 estados, 2 transiciones, 1 ticket, 1 admin), sin dependencia del orden de ejecución.
- **Cobertura focalizada:** las pruebas apuntan a la lógica de negocio, no a la capa HTTP. No se prueba el renderizado de plantillas ni la integración con Celery.

Deficiencia reconocida: la suite **no cubre el endpoint HTMX** de extremo a extremo. El protocolo `HX-Reswap` / `HX-Trigger`, el fragmento devuelto y el flujo de reordenamiento dentro de la misma columna quedan sin verificar automáticamente. Una prueba con `TestClient` sobre el endpoint completo detectaría regresiones en el contrato entre backend y frontend.

---

12. Limitaciones técnicas

Esta sección documenta honestamente las restricciones del módulo en su estado actual. Cada una debe considerarse antes de la incorporación.

12.1 Limitaciones funcionales

#	Limitación	Impacto	Mitigación
L1	**El tablero no captura el comentario obligatorio.** `tablero.html` envía `comentario: ''` de forma fija. Toda transición con `requiere_comentario=True` (cerrar, cancelar, bloquear, devolver) **siempre falla desde la interfaz**	Alto	Implementar diálogo `prompt()` o modal antes del `htmx.ajax`, condicionando si la transición de destino lo exige
L2	**La recarga en error recarga la página.** El rechazo usa `HX-Reswap: none` correctamente, pero el `setTimeout(location.reload, 1500)` recarga la página completa	Medio	Eliminar el `reload` si la reversión visual del DOM es fiable
L3	Sin CRUD completo de tickets	Bajo	Solo alta y movimiento; falta edición, borrado, búsqueda y filtrado
L4	Sin adjuntos ni comentarios	Bajo	Los tipos de evento `ADJUNTO_AGREGADO` y `COMENTARIO_AGREGADO` están declarados pero no emitidos
L5	Sin notificaciones reales	Bajo	`notificar_cambio_estado` solo escribe en auditoría
L6	Sin i18n	Bajo	Interfaz íntegramente en español

12.2 Limitaciones técnicas

#	Limitación	Impacto
T1	**Sin Alembic efectivo.** `alembic` está en dependencias pero el esquema se crea con `create_all()`, que **no altera tablas existentes**	Cambios de esquema requieren recrear la base
T2	**`app/schemas/` está vacío.** No hay modelos Pydantic de validación; los endpoints usan `Form` y diccionarios	Contratos de API no documentados automáticamente en OpenAPI
T3	**`_generar_codigo_ticket` tiene condición de carrera.** Lee `max(id)+1` sin bloqueo	Dos altas concurrentes pueden colisionar; la restricción UNIQUE hace fallar la segunda
T4	**Importaciones dinámicas con `__import__()`** en `auth.py:63` y `init_db.py:160`	Frágil ante refactorización; debería ser import directo
T5	**`cookie.secure` hardcodeado en `false`**	Inseguro bajo HTTPS
T6	**La suite de pruebas tarda ~154 s.** Celery intenta conectar a Redis y reintenta 20 veces por prueba	Requiere un fixture que desactive el broker en entorno de test
T7	`app/static/js/` existe pero está vacío	Todo el JS se sirve por CDN; sin activos propios
T8	Dependencias de frontend por CDN sin `integrity` (SRI)	Riesgo de compromiso de cadena de suministro

12.3 Hoja de ruta de maduración

Ordenado por relación esfuerzo/beneficio:

1. **Resolver L1** (captura de comentario) — desbloquea el flujo completo de cierre y cancelación
2. **Añadir pruebas del endpoint HTMX** — cierra el hueco de cobertura más importante
3. **Introducir Alembic** — requisito previo para cualquier evolución de esquema
4. **Parametrizar** `cookie.secure` y añadir rate limiting en login
5. **Migrar** `__import__()` a imports directos
6. **Modelos Pydantic** en `app/schemas/` — validación y documentación automática
7. **Notificaciones reales** (SMTP/webhook) sobre la base ya existente
8. **Internacionalización** de plantillas

---

13. Guía de incorporación a otros proyectos

Esta sección responde a la pregunta central: **cómo incorporar este módulo a un proyecto existente.**

13.1 Criterios de idoneidad

El módulo es adecuado si el proyecto destino cumple **al menos una** de estas condiciones:

- Gestiona **entidades con ciclo de vida** regulado por reglas (tickets, casos, órdenes, requisiciones, reportes)
- Requiere **trazabilidad de cambios** por auditoría o cumplimiento normativo
- Necesita **control de acceso por rol** con distintos niveles de privilegio
- Debe **calcular plazos** y detectar vencimientos (SLA)
- Busca una **vista Kanban** sin renunciar al control formal

13.2 Escenario A — Proyecto nuevo (adopción directa)

Esforzo estimado: bajo. El módulo funciona standalone.

```
# 1. Copiar el directorio itsm_kanban_mvp/ al proyecto
cp -r itsm_kanban_mvp/ mi-proyecto/

# 2. Instalar dependencias
cd mi-proyecto && pip install -r requirements.txt

# 3. Configurar entorno
cp .env.example .env
#   Editar SECRET_KEY, DATABASE_URL, BOOTSTRAP_ADMIN_PASSWORD

# 4. Crear esquema y datos iniciales
python -m app.db.init_db

# 5. Lanzar procesos
uvicorn app.main:app --host 0.0.0.0 --port 8000
celery -A app.core.celery_app:celery_app worker --loglevel=info
celery -A app.core.celery_app:celery_app beat   --loglevel=info
```

13.3 Escenario B — Proyecto existente (integración como subsistema)

Este es el escenario de mayor interés. El módulo aporta **cuatro activos reutilizables** con distinto grado de acoplamiento:

B.1 Núcleo de reglas ITSM — acoplamiento medio, alto valor

Activo: el motor de máquina de estados (`Transicion` + `mover_ticket`).

Desacoplamiento: el motor no depende de FastAPI. `mover_ticket()` recibe una `Session` de SQLAlchemy y un objeto `Usuario`; no importa nada del framework HTTP. Puede invocarse desde cualquier contexto.

Pasos de integración:

1. Crear las tablas `estados`, `transiciones`, `auditoria_eventos` en el esquema destino.
2. Sembrar el flujo de estados propio del proyecto destino.
3. Definir la matriz de transiciones según el proceso real.
4. Invocar `mover_ticket()` desde el servicio de dominio existente.

```
# Integración en un servicio de orders.py del proyecto destino
from app.services.kanban_service import mover_ticket, TransicionInvalidaError

@router.post("/ordenes/{id}/transicionar")
def transicionar(id: int, destino: int, usuario: Usuario, db: Session = Depends(get_db)):
    try:
        return mover_ticket(db, id, destino, usuario, comentario=...)
    except TransicionInvalidaError as e:
        raise HTTPException(422, e.detail)
```

Adaptación necesaria: renombrar el vocabulario. Si el dominio destino habla de *órdenes* en lugar de *tickets*, conviene renombrar `Ticket` → `Orden`, `AuditoriaEvento` → keep, `Transicion` → keep. El motor de reglas no cambia; solo la nomenclatura.

B.2 Patrón de auditoría append-only — acoplamiento cero, valor inmediato

Activo: el patrón de registro inmutable con taxonomía de eventos y desnormalización del actor.

Es una de las piezas más fáciles de adaptar y de mayor valor unitario. Puede aplicarse a **cualquier** entidad de cualquier proyecto sin modificar la estructura:

```
CREATE TABLE auditoria_eventos (
    id              SERIAL PRIMARY KEY,
    entidad_tipo    VARCHAR(50)   NOT NULL,   -- 'ticket', 'orden', 'factura'...
    entidad_id      INTEGER       NOT NULL,   -- genérico, sin FK dura
    tipo            VARCHAR(30)   NOT NULL,
    actor_id        INTEGER,                   -- FK SET NULL
    actor_username  VARCHAR(64)  NOT NULL,    -- desnormalizado
    valor_anterior  TEXT,
    valor_nuevo     TEXT,
    comentario      TEXT,
    created_at      TIMESTAMPTZ   NOT NULL DEFAULT now()
);
CREATE INDEX ix_auditoria ON auditoria_eventos (entidad_tipo, entidad_id, created_at);
```

El índice compuesto es indispensable: toda consulta de auditoría filtra por entidad y ordena por fecha.

Recomendación: priorizar este componente si el proyecto destino tiene requisitos de auditoría pero no ha implementado el patrón. Es una incorporación de bajo riesgo y alto retorno.

B.3 Celery + Redis para SLA y notificaciones — acoplamiento medio

Activo: la configuración Celery robusta y las tareas de recálculo.

El valor no está en el código específico sino en los **parámetros de robustez ya resueltos**: `acks_late`, `reject_on_worker_lost`, `prefetch_multiplier=1`, `soft_time_limit`, `time_limit`, `broker_connection_retry_on_startup`. Son problemas operables con todo proyecto que use Celery en producción, y su valor acumulado supera el del código de negocio específico.

B.4 Patrón UI Kanban + HTMX — acoplamiento alto, acoplamiento de contrato

Activo: la integración SortableJS + HTMX con protocolo de error por encabezados.

Precaución: es el componente **menos reutilizable tal cual**, por dos razones:

1. El contrato de datos está acoplado a los nombres del MVP (`data-ticket-id`, `data-estado-id`, endpoint `/api/v1/kanban/tickets/{id}/mover`).
2. Contiene la limitación L1 (comentario no capturado), que debe corregirse antes de replicar el patrón.

Lo reutilizable es el **patrón arquitectónico**, no el código:

Decisión	Valor
El DOM se mueve optimísticamente; el servidor confirma	Latencia percibida nula
El servidor devuelve fragmentos, no JSON	Cero lógica de renderizado en el cliente
El error viaja por `HX-Reswap: none`	Sin librería de estado en el cliente
El cliente nunca decide si un movimiento es válido	Fuente única de verdad

Si el proyecto destino ya tiene un tablero Kanban —como ocurre con Bitácora GRM v2—, la recomendación es **portar la lógica de reglas al backend existente** en lugar de sustituir su frontend.

13.4 Matriz de esfuerzo de integración

Componente	Esfuerzo	Riesgo	Valor	Recomendación
Patrón de auditoría	Bajo	Bajo	Alto	**Adoptar** en cualquier proyecto con requisitos de trazabilidad
Motor de reglas ITSM	Medio	Medio	Alto	**Adoptar** si hay ciclo de vida regulado
Configuración Celery robusta	Bajo	Bajo	Medio-Alto	**Adoptar** si ya usa Celery
Patrón UI Kanban+HTMX	Medio	Medio	Medio	**Portar el patrón**, no el código
Modelos completos	Bajo	Bajo	Variable	Referencia de diseño de esquema

13.5 Verificaciones previas a la incorporación

Antes de integrar, comprobar:

- [ ] ¿El proyecto destino tiene entidades con ciclo de vida regulado? *(si no, B.1 no aporta valor)*
- [ ] ¿Existe conflicto de nombres de tabla? `usuarios`, `estados`, `tickets` son nombres genéricos
- [ ] ¿Se usa SQLAlchemy 2.0? Requisito para reutilizar los modelos sin reescritura
- [ ] ¿Existe ya un sistema de auditoría? Evaluar **duplicación** antes de añadir otro
- [ ] ¿El proyecto tiene proceso de migraciones? Alembic vs. `create_all()` es una decisión de arquitectura
- [ ] ¿Se acepta el coste de dos procesos Celery adicionales? (worker + beat)

---

14. Sincronización con el proyecto anfitrión

En el caso específico de **Bitácora GRM v2** (repositorio anfitrión), coexisten dos tableros Kanban:

Aspecto	Tablero de Bitácora (`app/templates/kanban/`)	Tablero ITSM MVP (`itsm_kanban_mvp/`)
Madurez	En producción, historial extenso de correcciones	MVP, 4 pruebas
Lógica de transición	En el tablero principal	**Aislada y testeada** en `kanban_service`
Matriz de reglas	No externalizada	**Tabla `transiciones`**
Auditoría	Parcial	**Append-only garantizada**
Dependencias	fastapi 0.115.0, pydantic 2.9.2	fastapi 0.115.5, pydantic 2.10.3

Recomendación de convergencia: portar el motor de reglas ITSM al tablero existente, en lugar de mantener dos tableros. El tablero de Bitácora tiene la experiencia de producción —manejo de errores, estados terminales post-Cerrado, casos límite de producción— que el MVP no posee. El MVP aporta la disciplina de reglas y la separación servicio/endpoint que el tablero existente no tiene.

Pasos sugeridos para la convergencia:

1. Extraer `kanban_service.mover_ticket()` como módulo independiente sin dependencias de FastAPI.
2. Mapear las reglas ITSM sobre los estados reales de Bitácora (que incluye estados como «Producción OK» no contemplados en el MVP).
3. Sustituir la lógica de drag-drop actual del tablero de Bitácora por la llamada al servicio.
4. Unificar versiones de dependencias (`fastapi`, `pydantic`, `httpx`) en un solo archivo.

---

15. Resumen ejecutivo para decisión de incorporación

Qué es: un módulo de incidencias de ~2.000 líneas que implementa gestión ITSM sobre Kanban, con máquina de estados persistida, auditoría inmutable y procesamiento asíncrono de SLAs.

Lo que aporta con mayor valor:

1. **La tabla `transiciones`.** Externalizar las reglas de flujo a la base de datos —con rol requerido, comentario obligatorio y banderas de control— convierte el flujo de trabajo en dato configurable en lugar de código. Modificar el proceso no requiere desplgar.
1. **El patrón de auditoría append-only** con taxonomía de eventos y actor desnormalizado. Simple de adoptar, aplicable a cualquier entidad, e independiente de FastAPI.
1. **La separación validación-persistencia-notificación**, donde la notificación asíncrona es tolerante a fallos y la escritura es atómica con su rastro.

Lo que requiere trabajo antes de producción:

Prioridad	Ítem
1	Captura del comentario obligatorio en el tablero (L1)
2	Pruebas del endpoint HTMX
3	Migraciones Alembic
4	Parametrizar `cookie.secure` y añadir rate limiting
5	Modelos Pydantic en `app/schemas/`

Conclusión: el módulo es **adoptable** como núcleo de gestión de ciclo de vida. La mayor parte de su valor —motor de reglas, patrón de auditoría, configuración Celery— es código foundational reutilizable en proyectos con dominios distintos. El componente de interfaz requiere adaptación del contrato de datos y corrección de la limitación L1 antes de replicarse.

Documento generado tras verificación directa del código fuente del directorio `itsm_kanban_mvp/`. Las cifras de líneas, los conteos de entidades, de endpoints y de tareas, y todas las afirmaciones sobre comportamiento reflejan el estado verificado del código en la fecha indicada en el encabezado.

Verificaciones ejecutadas sobre el código: conteo de rutas por inspección de los decoradores `@router` y `@app`; contraste de versiones de dependencias contra `requirements.txt`; localización de las importaciones dinámicas `__import__()`; confirmación de la existencia de las excepciones `TransicionInvalidaError` y `PermisoInsuficienteError`; validación estructural de las 32 tablas del documento (celdas completas y consistentes); y escaneo de caracteres no previstos en el texto completo.